

Fiche Technique : Webpick Initializr

Générateur Dynamique de Projets et d'Infrastructure CI/CD

Abderrahman Bouanani
Étudiant Ingénieur Stagiaire dans Webpick

Juillet 2026

1. Présentation Générale

Problématique : La phase d'initialisation d'un nouveau projet client (le "Day 0") nécessite une configuration manuelle répétitive : création de dépôts, paramétrage des bases de données, configuration des pipelines CI/CD, et écriture des Dockerfiles. Cela entraîne des pertes de temps et des incohérences architecturales.

Solution - "Webpick Initializr" : Inspirée des générateurs modernes, cette solution interne permet via un formulaire web détaillé de générer dynamiquement l'intégralité de la base de code d'un projet, incluant la logique métier, l'infrastructure et les pipelines de déploiement.

2. État de l'Art et Comparatif

L'analyse des solutions existantes sur le marché permet de positionner la valeur ajoutée de cet outil sur-mesure pour l'entreprise.

Solution	Avantages / Inconvénients	Différence avec Webpick Initializr
Spring Initializr	Référence pour l'écosystème Java. Rapide et efficace. <i>Limite :</i> Limité à Spring Boot, n'inclut pas la CI/CD interne ni les autres frameworks de l'agence.	Génère des architectures multi-langages adaptées aux standards de Webpick, incluant les manifestes Docker et les pipelines Jenkins.
Yeoman	Outil CLI flexible avec un écosystème de générateurs. <i>Limite :</i> Utilisation en ligne de commande, courbe d'apprentissage, écosystème vieillissant.	Interface graphique intuitive (Frontend) éliminant la complexité des lignes de commande pour les nouvelles recrues.
Spotify Backstage	Plateforme DevOps extrêmement complète. <i>Limite :</i> Complexité d'installation, maintenance lourde, YAML intensif.	Approche "MVP" légère et pragmatique, codée de zéro, nécessitant zéro coût d'infrastructure supplémentaire et dédiée au flux de Webpick.

3. Architecture Technique

Le système est conçu autour d'une architecture garantissant robustesse et maintenabilité, en respectant les principes SOLID et une séparation nette des préoccupations (Clean Architecture).

- **Frontend (Interface Dynamique)** : Application en Angular ou React. Formulaire conditionnel intelligent adaptant les choix (ex : sélectionner "Spring Boot" affiche les options "Spring Security" ou "JPA").
- **Backend (Moteur Core)** : Développé en Spring Boot ou Python. Responsable de la gestion des requêtes JSON et de l'orchestration des tâches.
- **Moteur de Templating** : Utilisation d'un moteur (ex : FreeMarker, Thymeleaf ou Jinja2) pour la compilation dynamique des fichiers à partir de modèles "à trous" plutôt que par simple clonage statique.
- **Module DevOps** : Intégration d'outils de conteneurisation (Docker) et génération de pipelines Jenkins pré-configurés pour l'intégration continue.

4. Mécanique Interne de Génération

1. **Réception des données** : Le moteur reçoit les spécifications techniques (stack, base de données MySQL/MongoDB, type de CI/CD).
2. **Résolution des modèles** : Sélection des templates correspondants dans le catalogue interne.
3. **Compilation dynamique** : Injection des variables métiers dans le code source et les configurations d'infrastructure.
4. **Livrable** : Assemblage d'une archive (.zip) ou création directe du dépôt sur le gestionnaire de versions (Git/GitHub).

5. Bénéfices Stratégiques

- ✓ **Standardisation absolue** : Chaque projet démarre avec la même base, facilitant la réaffectation des ressources.
- ✓ **Productivité (Time-to-Market)** : La configuration initiale (Day 0) passe de plusieurs heures à quelques secondes.
- ✓ **Évolutivité** : Possibilité d'ajouter de nouvelles technologies au catalogue sans refactoriser le cœur du moteur de génération.